AI Agent 전문 개발 과정

Day 1 — Agent 문제 정의 & LLM 설계 전략

대상: AI 개발자 · 데이터 엔지니어 · 기술 리더

오늘의 일정

오늘의 일정

- **Session 1 (09:00–11:00)** — Agent 문제 정의와 과제 도출

오늘의 일정

- **Session 1 (09:00–11:00)** — Agent 문제 정의와 과제 도출
- **Session 2 (11:00–13:00)** — LLM 동작 원리 및 프롬프트 전략

오늘의 일정

- **Session 1 (09:00–11:00)** — Agent 문제 정의와 과제 도출
- **Session 2 (11:00–13:00)** — LLM 동작 원리 및 프롬프트 전략
- **점심 (13:00–14:00)**

오늘의 일정

- **Session 1 (09:00–11:00)** — Agent 문제 정의와 과제 도출
- **Session 2 (11:00–13:00)** — LLM 동작 원리 및 프롬프트 전략
- **점심 (13:00–14:00)**
- **Session 3 (14:00–16:00)** — Agent 기획서 구조화

오늘의 일정

- **Session 1 (09:00–11:00)** — Agent 문제 정의와 과제 도출
- **Session 2 (11:00–13:00)** — LLM 동작 원리 및 프롬프트 전략
- **점심 (13:00–14:00)**
- **Session 3 (14:00–16:00)** — Agent 기획서 구조화
- **Session 4 (16:00–18:00)** — MCP · RAG · Hybrid 구조 판단

오늘의 일정

- **Session 1 (09:00–11:00)** — Agent 문제 정의와 과제 도출
- **Session 2 (11:00–13:00)** — LLM 동작 원리 및 프롬프트 전략
- **점심 (13:00–14:00)**
- **Session 3 (14:00–16:00)** — Agent 기획서 구조화
- **Session 4 (16:00–18:00)** — MCP · RAG · Hybrid 구조 판단

> 강의 30% · 실습 70% — 모든 실습은 I DO → WE DO → YOU DO

Session 1

Agent 문제 정의와 과제 도출

Agent가 실패하는 가장 큰 이유

Agent가 실패하는 가장 큰 이유

> "왜 Agent가 필요한지" 정의하지 않고 시작하기 때문이다

Agent가 실패하는 가장 큰 이유

> "왜 Agent가 필요한지" 정의하지 않고 시작하기 때문이다

- 기술에서 시작 → 문제를 기술에 끼워 맞춤

Agent가 실패하는 가장 큰 이유

> "왜 Agent가 필요한지" 정의하지 않고 시작하기 때문이다

- 기술에서 시작 → 문제를 기술에 끼워 맞춤
- 성공 기준 없음 → "잘 되는 것 같다"로 마무리

Agent가 실패하는 가장 큰 이유

> "왜 Agent가 필요한지" 정의하지 않고 시작하기 때문이다

- 기술에서 시작 → 문제를 기술에 끼워 맞춤
- 성공 기준 없음 → "잘 되는 것 같다"로 마무리
- 실패 케이스 미정의 → 런타임에 놀라게 됨

Agent에 적합한 문제

Agent에 적합한 문제

- **다단계 추론** — 여러 단계를 순차적으로 처리

Agent에 적합한 문제

- **다단계 추론** — 여러 단계를 순차적으로 처리
- **도구 사용** — 외부 API, DB, 파일 시스템 접근

Agent에 적합한 문제

- **다단계 추론** — 여러 단계를 순차적으로 처리
- **도구 사용** — 외부 API, DB, 파일 시스템 접근
- **비결정적 흐름** — 상황에 따라 다음 단계가 달라짐

Agent에 적합한 문제

- **다단계 추론** — 여러 단계를 순차적으로 처리
- **도구 사용** — 외부 API, DB, 파일 시스템 접근
- **비결정적 흐름** — 상황에 따라 다음 단계가 달라짐
- **반복 자동화** — 사람이 하면 지루하고 느린 작업

Agent에 적합한 문제

- **다단계 추론** — 여러 단계를 순차적으로 처리
- **도구 사용** — 외부 API, DB, 파일 시스템 접근
- **비결정적 흐름** — 상황에 따라 다음 단계가 달라짐
- **반복 자동화** — 사람이 하면 지루하고 느린 작업

Agent가 과잉인 경우

- 단순 Q&A → 챗봇 or RAG
- 고정 템플릿 → 프롬프트
- 단일 API 호출 → 직접 호출

Agent가 필요한 경우

- 이슈 분류 + 담당자 지정 + 알림
- 데이터 수집 + 분석 + 보고서
- 조건별 분기 + 예외 처리

Pain → Task → Skill → Tool

```
Pain (고통) → Task (과제) → Skill (능력) → Tool (도구)
```

단계	내용

Pain → Task → Skill → Tool

```
Pain (고통) → Task (과제) → Skill (능력) → Tool (도구)
```

예시: 주간 보고서 작성이 너무 오래 걸린다

단계	내용

Pain → Task → Skill → Tool

```
Pain (고통) → Task (과제) → Skill (능력) → Tool (도구)
```

예시: 주간 보고서 작성이 너무 오래 걸린다

단계	내용
Pain	매주 3시간을 보고서 수동 작성에 낭비

Pain → Task → Skill → Tool

```
Pain (고통) → Task (과제) → Skill (능력) → Tool (도구)
```

예시: 주간 보고서 작성이 너무 오래 걸린다

단계	내용
Pain	매주 3시간을 보고서 수동 작성에 낭비
Task	여러 시스템 데이터를 모아 요약 보고서 생성

Pain → Task → Skill → Tool

```
Pain (고통) → Task (과제) → Skill (능력) → Tool (도구)
```

예시: 주간 보고서 작성이 너무 오래 걸린다

단계	내용
Pain	매주 3시간을 보고서 수동 작성에 낭비
Task	여러 시스템 데이터를 모아 요약 보고서 생성
Skill	데이터 수집, 수치 해석, 문서 작성, 발송

Pain → Task → Skill → Tool

```
Pain (고통) → Task (과제) → Skill (능력) → Tool (도구)
```

예시: 주간 보고서 작성이 너무 오래 걸린다

단계	내용
Pain	매주 3시간을 보고서 수동 작성에 낭비
Task	여러 시스템 데이터를 모아 요약 보고서 생성
Skill	데이터 수집, 수치 해석, 문서 작성, 발송
Tool	Jira API, DB 쿼리, LLM 요약, 슬랙 발송

Agent 3가지 패턴

Agent 3가지 패턴

자동화형

- 트리거 → 처리 → 저장
- 흐름이 고정됨
- 예: 일간 보고서 발송

Agent 3가지 패턴

자동화형

- 트리거 → 처리 → 저장
- 흐름이 고정됨
- 예: 일간 보고서 발송

분석형

- 질문 → 탐색 → 설명
- 데이터 소스 다양
- 예: 이탈 고객 분석

Agent 3가지 패턴

자동화형

- 트리거 → 처리 → 저장
- 흐름이 고정됨
- 예: 일간 보고서 발송

분석형

- 질문 → 탐색 → 설명
- 데이터 소스 다양
- 예: 이탈 고객 분석

Planner형

- 목표 → 계획 → 실행
- 흐름이 동적
- 예: 경쟁사 분석 보고서

Agent 3가지 패턴

자동화형

- 트리거 → 처리 → 저장
- 흐름이 고정됨
- 예: 일간 보고서 발송

분석형

- 질문 → 탐색 → 설명
- 데이터 소스 다양
- 예: 이탈 고객 분석

Planner형

- 목표 → 계획 → 실행
- 흐름이 동적
- 예: 경쟁사 분석 보고서

난이도: 자동화형 < 분석형 < Planner형 · 비용도 동일 순서

RAG vs Agent 판단 기준

RAG 선택

- 단순히 "찾아서 대답"하면 됨
- 문서 기반 지식만 필요
- 외부 시스템 조작 불필요
- 예: 사내 문서 Q&A

Agent 선택

- 외부 시스템과 상호작용 필요
- 결과 보고 다음 행동이 달라짐
- 여러 단계 처리 필요
- 예: 이슈 자동 처리

RAG vs Agent 판단 기준

RAG 선택

- 단순히 "찾아서 대답"하면 됨
- 문서 기반 지식만 필요
- 외부 시스템 조작 불필요
- 예: 사내 문서 Q&A

Agent 선택

- 외부 시스템과 상호작용 필요
- 결과 보고 다음 행동이 달라짐
- 여러 단계 처리 필요
- 예: 이슈 자동 처리

> RAG와 Agent는 배타적이지 않다. Agent가 RAG를 도구로 사용하는 Hybrid도 많다.

퀴즈: Agent 적합성 판단

다음 중 AI Agent 적용이 가장 적합한 시나리오는?

A) 사내 FAQ 문서에서 답변 찾기

B) 고정 HTML 템플릿에 데이터 채우기

C) 이슈 분석 + 담당자 지정 + 알림 발송

D) 텍스트를 영어에서 한국어로 번역

퀴즈: Agent 적합성 판단

다음 중 AI Agent 적용이 가장 적합한 시나리오는?

A) 사내 FAQ 문서에서 답변 찾기

B) 고정 HTML 템플릿에 데이터 채우기

C) 이슈 분석 + 담당자 지정 + 알림 발송

D) 텍스트를 영어에서 한국어로 번역

정답: **C** — 다단계 처리 + 외부 시스템 조작 + 비결정적 흐름

Session 1 실습

I DO → WE DO → YOU DO

실습: Agent 후보 도출

실습: Agent 후보 도출

- **I DO (15분)**: 강사가 실제 업무를 Pain → Tool로 분해

실습: Agent 후보 도출

- **I DO (15분)**: 강사가 실제 업무를 Pain → Tool로 분해
- **WE DO (20분)**: 수강생이 자신의 업무 1개를 함께 분해

실습: Agent 후보 도출

- **I DO (15분)**: 강사가 실제 업무를 Pain → Tool로 분해
- **WE DO (20분)**: 수강생이 자신의 업무 1개를 함께 분해
- **YOU DO (25분)**: 개인 업무 기반 Agent 후보 2개 도출

실습: Agent 후보 도출

- **I DO (15분)**: 강사가 실제 업무를 Pain → Tool로 분해
- **WE DO (20분)**: 수강생이 자신의 업무 1개를 함께 분해
- **YOU DO (25분)**: 개인 업무 기반 Agent 후보 2개 도출

YOU DO 제출물

- Agent 후보 2개의 Pain → Task → Skill → Tool
- 각각의 패턴 분류 (자동화형/분석형/Planner형)
- RAG vs Agent 선택 이유

Session 2

LLM 동작 원리 및 프롬프트 전략 심화

Token과 Context Window

Token과 Context Window

- **Token**: LLM이 처리하는 최소 단위 (단어·글자 조각)

Token과 Context Window

- **Token**: LLM이 처리하는 최소 단위 (단어·글자 조각)
- **한국어**는 영어보다 토큰 효율이 낮음

Token과 Context Window

- **Token**: LLM이 처리하는 최소 단위 (단어·글자 조각)
- **한국어**는 영어보다 토큰 효율이 낮음
- **Context Window**: 한 번에 처리 가능한 최대 토큰 수

Token과 Context Window

- **Token**: LLM이 처리하는 최소 단위 (단어·글자 조각)
- **한국어**는 영어보다 토큰 효율이 낮음
- **Context Window**: 한 번에 처리 가능한 최대 토큰 수

```
Context Window 구성:
├── System Prompt     : 500 토큰
├── 대화 이력         : 2,000 토큰
├── 현재 사용자 입력  : 300 토큰
└── 출력 (Max Tokens) : 1,000 토큰
    = 총 3,800 토큰 사용
```

Token과 Context Window

- **Token**: LLM이 처리하는 최소 단위 (단어·글자 조각)
- **한국어**는 영어보다 토큰 효율이 낮음
- **Context Window**: 한 번에 처리 가능한 최대 토큰 수

```
Context Window 구성:
├── System Prompt     : 500 토큰
├── 대화 이력         : 2,000 토큰
├── 현재 사용자 입력  : 300 토큰
└── 출력 (Max Tokens) : 1,000 토큰
    = 총 3,800 토큰 사용
```

> Context 초과 시: 앞부분이 잘리거나 에러 발생 → RAG 검색 결과도 포함됨에 주의

Hallucination 이해

Hallucination 이해

Hallucination: LLM이 사실이 아닌 내용을 자신감 있게 생성하는 현상

Hallucination 이해

> **Hallucination**: LLM이 사실이 아닌 내용을 자신감 있게 생성하는 현상

자주 발생하는 상황

Hallucination 이해

> **Hallucination**: LLM이 사실이 아닌 내용을 자신감 있게 생성하는 현상

자주 발생하는 상황

- 학습 데이터 이후의 최신 정보 요청
- 구체적 수치, 날짜, 인용 요청
- 너무 열린 질문 (제약 없음)
- 모델이 잘 모르는 도메인 세부사항

Hallucination 완화 전략

Hallucination 완화 전략

RAG 결합 관련 문서를 Context에 포함 "제공된 문서만 근거로 사용하라"

Hallucination 완화 전략

RAG 결합 관련 문서를 Context에 포함 "제공된 문서만 근거로 사용하라"

불확실성 명시 System Prompt에 추가: "모르면 모른다고 답하라"

Hallucination 완화 전략

RAG 결합 관련 문서를 Context에 포함 "제공된 문서만 근거로 사용하라"

불확실성 명시 System Prompt에 추가: "모르면 모른다고 답하라"

Structured Output JSON Schema로 응답 범위 제한 임의 생성 공간 축소

Hallucination 완화 전략

RAG 결합 관련 문서를 Context에 포함 "제공된 문서만 근거로 사용하라"

불확실성 명시 System Prompt에 추가: "모르면 모른다고 답하라"

Structured Output JSON Schema로 응답 범위 제한 임의 생성 공간 축소

Agent 설계 원칙: "LLM이 틀릴 수 있다"는 전제로 검증 로직을 항상 포함하라

프롬프트 전략: Zero-shot

프롬프트 전략: Zero-shot

지시만 제공하고 예시 없이 수행을 요청한다.

프롬프트 전략: Zero-shot

지시만 제공하고 예시 없이 수행을 요청한다.

```
prompt = """
다음 고객 리뷰를 긍정/부정/중립으로 분류하라.
리뷰: "배송이 빠르긴 했는데 포장이 엉망이었어요."
"""
```

프롬프트 전략: Zero-shot

지시만 제공하고 예시 없이 수행을 요청한다.

```
prompt = """
다음 고객 리뷰를 긍정/부정/중립으로 분류하라.
리뷰: "배송이 빠르긴 했는데 포장이 엉망이었어요."
"""
```

장점: 프롬프트 짧음 · 토큰 효율 좋음

단점: 복잡한 작업에서 품질 저하

프롬프트 전략: Zero-shot

지시만 제공하고 예시 없이 수행을 요청한다.

```
prompt = """
다음 고객 리뷰를 긍정/부정/중립으로 분류하라.
리뷰: "배송이 빠르긴 했는데 포장이 엉망이었어요."
"""
```

장점: 프롬프트 짧음 · 토큰 효율 좋음

단점: 복잡한 작업에서 품질 저하

적합: 단순 분류, 번역, 요약

프롬프트 전략: Few-shot

프롬프트 전략: Few-shot

작업 방식을 예시(shot)로 보여준다.

프롬프트 전략: Few-shot

작업 방식을 예시(shot)로 보여준다.

```
prompt = """
리뷰: "정말 좋아요!" → 긍정
리뷰: "완전 실망입니다." → 부정
리뷰: "그냥 평범해요." → 중립

리뷰: "배송이 빠르긴 했는데 포장이 엉망이었어요." →
"""
```

프롬프트 전략: Few-shot

작업 방식을 예시(shot)로 보여준다.

```
prompt = """
리뷰: "정말 좋아요!" → 긍정
리뷰: "완전 실망입니다." → 부정
리뷰: "그냥 평범해요." → 중립

리뷰: "배송이 빠르긴 했는데 포장이 엉망이었어요." →
"""
```

장점: 출력 형식·품질 안정적

단점: 토큰 사용량 증가

프롬프트 전략: Chain-of-Thought

프롬프트 전략: Chain-of-Thought

단계적 추론 과정을 포함하도록 유도한다.

프롬프트 전략: Chain-of-Thought

단계적 추론 과정을 포함하도록 유도한다.

```
# 마법의 한 문장
prompt = """
보험 청구서를 분석하여 유효성을 판단하라.

단계적으로 생각해보자.
"""
```

프롬프트 전략: Chain-of-Thought

단계적 추론 과정을 포함하도록 유도한다.

```
# 마법의 한 문장
prompt = """
보험 청구서를 분석하여 유효성을 판단하라.

단계적으로 생각해보자.
"""
```

장점: 복잡한 추론 정확도 대폭 향상

단점: 출력 토큰 증가 → 비용·지연 상승

프롬프트 전략: Chain-of-Thought

단계적 추론 과정을 포함하도록 유도한다.

```
# 마법의 한 문장
prompt = """
보험 청구서를 분석하여 유효성을 판단하라.

단계적으로 생각해보자.
"""
```

장점: 복잡한 추론 정확도 대폭 향상

단점: 출력 토큰 증가 → 비용·지연 상승

적합: 수학 문제, 논리 추론, 복잡한 판단

프롬프트 전략 비교

전략	토큰	정확도	속도	적합 상황
Zero-shot	낮음	보통	빠름	단순 작업
Few-shot	중간	높음	중간	형식 중요
CoT	높음	높음	느림	복잡 추론
CoT + Few-shot	매우 높음	매우 높음	느림	고정확도

프롬프트 전략 비교

전략	토큰	정확도	속도	적합 상황
Zero-shot	낮음	보통	빠름	단순 작업
Few-shot	중간	높음	중간	형식 중요
CoT	높음	높음	느림	복잡 추론
CoT + Few-shot	매우 높음	매우 높음	느림	고정확도

선택 가이드

- 비용 제약 → Zero-shot 먼저 시도
- 형식 중요 → Few-shot
- 추론 필요 → CoT

Structured Output

Structured Output

Agent에서 LLM 출력은 다음 로직의 **입력**이 된다.
자연어 파싱은 불안정하다 → JSON Schema로 형식을 강제하라.

Structured Output

> Agent에서 LLM 출력은 다음 로직의 **입력**이 된다.
> 자연어 파싱은 불안정하다 → JSON Schema로 형식을 강제하라.

```
# Tool Use 방식 (가장 안정적)
tools = [{
    "name": "classify_review",
    "input_schema": {
        "type": "object",
        "properties": {
            "category": {"type": "string", "enum": ["긍정", "부정", "중립"]},
            "confidence": {"type": "number"},
            "reason": {"type": "string"}
        },
        "required": ["category", "confidence", "reason"]
    }
}]
```

Structured Output

Agent에서 LLM 출력은 다음 로직의 **입력**이 된다.
자연어 파싱은 불안정하다 → JSON Schema로 형식을 강제하라.

```
# Tool Use 방식 (가장 안정적)
tools = [{
    "name": "classify_review",
    "input_schema": {
        "type": "object",
        "properties": {
            "category": {"type": "string", "enum": ["긍정", "부정", "중립"]},
            "confidence": {"type": "number"},
            "reason": {"type": "string"}
        },
        "required": ["category", "confidence", "reason"]
    }
}]
```

Tool Use로 강제하면 스키마 위반 응답이 거의 없다

비용·Latency 최적화

비용·Latency 최적화

- **1순위**: 모델 선택 — 작업에 맞는 모델 사용

비용·Latency 최적화

- **1순위**: 모델 선택 — 작업에 맞는 모델 사용
- **2순위**: 출력 길이 제한 — max_tokens 적절히 설정

비용·Latency 최적화

- **1순위**: 모델 선택 — 작업에 맞는 모델 사용
- **2순위**: 출력 길이 제한 — max_tokens 적절히 설정
- **3순위**: 입력 압축 — 불필요한 Context 제거

비용·Latency 최적화

- **1순위**: 모델 선택 — 작업에 맞는 모델 사용
- **2순위**: 출력 길이 제한 — max_tokens 적절히 설정
- **3순위**: 입력 압축 — 불필요한 Context 제거
- **4순위**: 캐싱 활용 — 반복 System Prompt 캐시

비용·Latency 최적화

- **1순위**: 모델 선택 — 작업에 맞는 모델 사용
- **2순위**: 출력 길이 제한 — max_tokens 적절히 설정
- **3순위**: 입력 압축 — 불필요한 Context 제거
- **4순위**: 캐싱 활용 — 반복 System Prompt 캐시

Latency 단축

- 병렬 API 호출
- 스트리밍 응답
- 작은 모델 우선

비용 공식

```
비용 = 입력토큰 × 단가
     + 출력토큰 × 단가
(출력 단가 > 입력 단가)
```

퀴즈: 프롬프트 전략 선택

보험 청구서 유효성 판단 — 단계별 근거 설명 필요, 실수 허용 낮음

어떤 전략이 가장 적합한가?

A) Zero-shot

B) Few-shot

C) CoT + Few-shot

D) 단순 LLM 호출

퀴즈: 프롬프트 전략 선택

보험 청구서 유효성 판단 — 단계별 근거 설명 필요, 실수 허용 낮음

어떤 전략이 가장 적합한가?

A) Zero-shot

B) Few-shot

C) CoT + Few-shot

D) 단순 LLM 호출

정답: **C** — 단계별 추론(CoT) + 일관된 형식(Few-shot) 조합

Session 2 실습

프롬프트 전략별 응답 비교

실습: 전략 비교 코드

실습: 전략 비교 코드

- **I DO (15분)**: 강사가 Zero/Few/CoT를 같은 태스크로 비교

실습: 전략 비교 코드

- **I DO (15분)**: 강사가 Zero/Few/CoT를 같은 태스크로 비교
- **WE DO (20분)**: Structured Output (Tool Use) 함께 구현

실습: 전략 비교 코드

- **I DO (15분)**: 강사가 Zero/Few/CoT를 같은 태스크로 비교
- **WE DO (20분)**: Structured Output (Tool Use) 함께 구현
- **YOU DO (25분)**: 자신의 태스크에 세 전략 적용 + 토큰·지연 비교

실습: 전략 비교 코드

- **I DO (15분)**: 강사가 Zero/Few/CoT를 같은 태스크로 비교
- **WE DO (20분)**: Structured Output (Tool Use) 함께 구현
- **YOU DO (25분)**: 자신의 태스크에 세 전략 적용 + 토큰·지연 비교

YOU DO 제출물 (`labs/prompt-strategy-comparison/src/`)

- 동일 태스크에 3가지 전략 코드
- 토큰 수 · 지연 · 품질 비교 결과
- Structured Output (JSON) 구현

Session 3

Agent 기획서 구조화

기획서가 필요한 이유

기획서가 필요한 이유

- 개발자: 개발 범위 명확화

기획서가 필요한 이유

- 개발자: 개발 범위 명확화
- QA: 테스트 케이스 도출

기획서가 필요한 이유

- 개발자: 개발 범위 명확화
- QA: 테스트 케이스 도출
- 운영자: 모니터링 포인트 파악

기획서가 필요한 이유

- 개발자: 개발 범위 명확화
- QA: 테스트 케이스 도출
- 운영자: 모니터링 포인트 파악
- 팀 전체: 커뮤니케이션 도구

기획서가 필요한 이유

- 개발자: 개발 범위 명확화
- QA: 테스트 케이스 도출
- 운영자: 모니터링 포인트 파악
- 팀 전체: 커뮤니케이션 도구

> "어디서 시작해서 어디서 끝나는지" 모르면
> 테스트도 못하고 예외 처리도 할 수 없다

Task → Sub-task 분해 원칙

Task → Sub-task 분해 원칙

- 하나의 Sub-task = 하나의 책임

Task → Sub-task 분해 원칙

- 하나의 Sub-task = 하나의 책임
- 독립적으로 테스트 가능해야 함

Task → Sub-task 분해 원칙

- 하나의 Sub-task = 하나의 책임
- 독립적으로 테스트 가능해야 함
- Sub-task 출력 = 다음 Sub-task 입력

Task → Sub-task 분해 원칙

- 하나의 Sub-task = 하나의 책임
- 독립적으로 테스트 가능해야 함
- Sub-task 출력 = 다음 Sub-task 입력

```
주간 보고서 Agent:
ST-1: 데이터 수집 (Jira API, DB 쿼리)
  ↓
ST-2: 데이터 정제 (원시 → 표준 포맷)
  ↓
ST-3: 요약 생성 (LLM)
  ↓
ST-4: 배포 (슬랙, 이메일)
```

Stateless vs Stateful

Stateless

- 각 호출이 독립적
- 이전 실행을 기억 안 함
- 수평 확장 쉬움
- 재시도 안전
- 예: 티켓 분류, 번역

Stateful

- 이전 실행 결과 기억
- 단계적 처리 가능
- 대화형 상호작용 지원
- 체크포인트 필요
- 예: 리서치 Agent

Stateless vs Stateful

Stateless

- 각 호출이 독립적
- 이전 실행을 기억 안 함
- 수평 확장 쉬움
- 재시도 안전
- 예: 티켓 분류, 번역

Stateful

- 이전 실행 결과 기억
- 단계적 처리 가능
- 대화형 상호작용 지원
- 체크포인트 필요
- 예: 리서치 Agent

선택 기준: 이전 결과가 다음 실행에 필요한가?
→ Yes: Stateful No: Stateless

Input – Process – Output 명세

Input – Process – Output 명세

모든 Sub-task는 I/O가 명확해야 한다.

Input – Process – Output 명세

모든 Sub-task는 I/O가 명확해야 한다.

```
@dataclass
class TicketInput:
    ticket_id: str
    content: str
    priority: Literal["low", "medium", "high"] = "medium"

@dataclass
class TicketOutput:
    category: Literal["bug", "feature", "question", "unclassified"]
    severity: int   # 1-5
    assignee: str
    confidence: float  # 0.0-1.0
```

Input – Process – Output 명세

모든 Sub-task는 I/O가 명확해야 한다.

```python
@dataclass
class TicketInput:
    ticket_id: str
    content: str
    priority: Literal["low", "medium", "high"] = "medium"

@dataclass
class TicketOutput:
    category: Literal["bug", "feature", "question", "unclassified"]
    severity: int   # 1-5
    assignee: str
    confidence: float  # 0.0-1.0
```

> Python dataclass나 Pydantic으로 명시하면 오류를 조기에 발견한다

예외 처리 전략

예외 처리 전략

- **AUTO_RETRY** — 네트워크 오류, 타임아웃 → 자동 재시도

예외 처리 전략

- **AUTO_RETRY** — 네트워크 오류, 타임아웃 → 자동 재시도
- **FALLBACK** — LLM 파싱 실패 → 기본값 사용

예외 처리 전략

- **AUTO_RETRY** — 네트워크 오류, 타임아웃 → 자동 재시도
- **FALLBACK** — LLM 파싱 실패 → 기본값 사용
- **NOTIFY** — 비정상 데이터 → 알림 발송

예외 처리 전략

- **AUTO_RETRY** — 네트워크 오류, 타임아웃 → 자동 재시도
- **FALLBACK** — LLM 파싱 실패 → 기본값 사용
- **NOTIFY** — 비정상 데이터 → 알림 발송
- **HUMAN_REVIEW** — 비즈니스 오류 → 사람 검토

예외 처리 전략

- **AUTO_RETRY** — 네트워크 오류, 타임아웃 → 자동 재시도
- **FALLBACK** — LLM 파싱 실패 → 기본값 사용
- **NOTIFY** — 비정상 데이터 → 알림 발송
- **HUMAN_REVIEW** — 비즈니스 오류 → 사람 검토
- **HALT** — 데이터 손상 → 전체 중단

예외 처리 전략

- **AUTO_RETRY** — 네트워크 오류, 타임아웃 → 자동 재시도
- **FALLBACK** — LLM 파싱 실패 → 기본값 사용
- **NOTIFY** — 비정상 데이터 → 알림 발송
- **HUMAN_REVIEW** — 비즈니스 오류 → 사람 검토
- **HALT** — 데이터 손상 → 전체 중단

> "외부 API 실패 · 데이터 오류 · LLM 응답 오류" — 이 세 범주만 커버해도 80%를 처리한다

퀴즈: Workflow 순서 오류

다음 Workflow에서 문제점은?

퀴즈: Workflow 순서 오류

다음 Workflow에서 문제점은?

```
ST-1: 보고서 생성 (LLM 요약)
ST-2: 데이터 수집 (DB 쿼리)
Workflow: ST-1 → ST-2
```

퀴즈: Workflow 순서 오류

다음 Workflow에서 문제점은?

```
ST-1: 보고서 생성 (LLM 요약)
ST-2: 데이터 수집 (DB 쿼리)
Workflow: ST-1 → ST-2
```

순서 오류: ST-1은 ST-2의 결과물이 필요하다.
올바른 순서: ST-2(데이터 수집) → ST-1(보고서 생성)

Session 3 실습

Agent 구조 다이어그램 설계

실습: 기획서 작성

실습: 기획서 작성

- **I DO (15분)**: Session 1 Agent 후보를 기획서로 구조화

실습: 기획서 작성

- **I DO (15분)**: Session 1 Agent 후보를 기획서로 구조화
- **WE DO (20분)**: Sub-task 3-5개로 분해, IPO 1개 완성

실습: 기획서 작성

- **I DO (15분)**: Session 1 Agent 후보를 기획서로 구조화
- **WE DO (20분)**: Sub-task 3-5개로 분해, IPO 1개 완성
- **YOU DO (25분)**: 전체 기획서 + Workflow 다이어그램 완성

실습: 기획서 작성

- **I DO (15분)**: Session 1 Agent 후보를 기획서로 구조화
- **WE DO (20분)**: Sub-task 3-5개로 분해, IPO 1개 완성
- **YOU DO (25분)**: 전체 기획서 + Workflow 다이어그램 완성

YOU DO 제출물 (`labs/agent-problem-definition/artifacts/`)

- Workflow 다이어그램 (텍스트 또는 mermaid)
- 모든 Sub-task IPO 명세
- Stateless/Stateful 선택 이유
- 실패 케이스 3개 이상

Session 4

MCP · RAG · Hybrid 구조 판단

MCP (Function Calling) 이해

MCP (Function Calling) 이해

> LLM이 직접 코드를 실행하는 것이 아니라
> **"이 Tool을 이 인자로 호출하라"는 명령을 생성**하고
> 실제 실행은 시스템이 담당한다

MCP (Function Calling) 이해

> LLM이 직접 코드를 실행하는 것이 아니라
> **"이 Tool을 이 인자로 호출하라"는 명령을 생성**하고
> 실제 실행은 시스템이 담당한다

```
사용자 요청 → LLM → Tool 호출 명령 생성
                          ↓
                     실제 Tool 실행
                          ↓
                     결과를 LLM에 전달
                          ↓
                     최종 응답 생성
```

좋은 Tool 설계 기준

좋은 Tool 설계 기준

- **단일 책임**: 하나의 Tool = 하나의 명확한 기능

좋은 Tool 설계 기준

- **단일 책임**: 하나의 Tool = 하나의 명확한 기능
- **명확한 Description**: LLM이 언제 이 Tool을 쓸지 이해

좋은 Tool 설계 기준

- **단일 책임**: 하나의 Tool = 하나의 명확한 기능
- **명확한 Description**: LLM이 언제 이 Tool을 쓸지 이해
- **명확한 에러 응답**: LLM이 이해하는 에러 메시지

좋은 Tool 설계 기준

- **단일 책임**: 하나의 Tool = 하나의 명확한 기능
- **명확한 Description**: LLM이 언제 이 Tool을 쓸지 이해
- **명확한 에러 응답**: LLM이 이해하는 에러 메시지

```python
# 나쁜 예시
def manage_everything(action: str, data: dict) -> dict:
    """모든 데이터 관리"""  # LLM이 언제 써야 할지 모름

# 좋은 예시
def get_user_by_id(user_id: str) -> dict:
    """사용자 ID로 프로필 정보를 조회한다"""
```

좋은 Tool 설계 기준

- **단일 책임**: 하나의 Tool = 하나의 명확한 기능
- **명확한 Description**: LLM이 언제 이 Tool을 쓸지 이해
- **명확한 에러 응답**: LLM이 이해하는 에러 메시지

```python
# 나쁜 예시
def manage_everything(action: str, data: dict) -> dict:
    """모든 데이터 관리"""  # LLM이 언제 써야 할지 모름

# 좋은 예시
def get_user_by_id(user_id: str) -> dict:
    """사용자 ID로 프로필 정보를 조회한다"""
```

Tool이 15개를 넘으면 LLM의 선택 정확도가 떨어진다
→ 그룹화하거나 Sub-agent로 분리하라

RAG 구조 흐름

RAG 구조 흐름

```
[색인 단계]
문서 수집 → 청킹 → 임베딩 → 벡터 DB 저장

[검색 단계]
사용자 질문 → 질문 임베딩 → 유사도 검색
           → 관련 청크 추출 → Context 구성
           → LLM 답변 생성
```

RAG 구조 흐름

```
[색인 단계]
문서 수집 → 청킹 → 임베딩 → 벡터 DB 저장

[검색 단계]
사용자 질문 → 질문 임베딩 → 유사도 검색
           → 관련 청크 추출 → Context 구성
           → LLM 답변 생성
```

- **청킹**: 문서를 적절한 크기로 분할 (500토큰 권장)

RAG 구조 흐름

```
[색인 단계]
문서 수집 → 청킹 → 임베딩 → 벡터 DB 저장

[검색 단계]
사용자 질문 → 질문 임베딩 → 유사도 검색
           → 관련 청크 추출 → Context 구성
           → LLM 답변 생성
```

- **청킹**: 문서를 적절한 크기로 분할 (500토큰 권장)
- **임베딩**: 텍스트 → 벡터 변환 (같은 모델 유지 필수)

RAG 구조 흐름

```
[색인 단계]
문서 수집 → 청킹 → 임베딩 → 벡터 DB 저장

[검색 단계]
사용자 질문 → 질문 임베딩 → 유사도 검색
           → 관련 청크 추출 → Context 구성
           → LLM 답변 생성
```

- **청킹**: 문서를 적절한 크기로 분할 (500토큰 권장)
- **임베딩**: 텍스트 → 벡터 변환 (같은 모델 유지 필수)
- **유사도 검색**: 질문과 가장 관련 있는 청크 추출

RAG 구조 흐름

```
[색인 단계]
문서 수집 → 청킹 → 임베딩 → 벡터 DB 저장

[검색 단계]
사용자 질문 → 질문 임베딩 → 유사도 검색
           → 관련 청크 추출 → Context 구성
           → LLM 답변 생성
```

- **청킹**: 문서를 적절한 크기로 분할 (500토큰 권장)
- **임베딩**: 텍스트 → 벡터 변환 (같은 모델 유지 필수)
- **유사도 검색**: 질문과 가장 관련 있는 청크 추출
- **chunk_overlap**: 청크 간 겹침으로 문맥 연결

Tool 중심 vs RAG 중심 비교

MCP (Tool 중심)

- 실시간 데이터 지원
- 외부 시스템 조작 가능
- Tool 수에 따라 복잡도 증가
- API 의존성 높음

RAG 중심

- 정적 지식 Q&A에 강함
- 근거 있는 답변 가능
- 실시간 데이터 미지원
- 검색 품질에 의존

Tool 중심 vs RAG 중심 비교

MCP (Tool 중심)

- 실시간 데이터 지원
- 외부 시스템 조작 가능
- Tool 수에 따라 복잡도 증가
- API 의존성 높음

RAG 중심

- 정적 지식 Q&A에 강함
- 근거 있는 답변 가능
- 실시간 데이터 미지원
- 검색 품질에 의존

판단 질문

- 실시간 데이터 필요한가? → MCP
- 외부 시스템 조작 필요한가? → MCP
- 정적 문서 기반 Q&A인가? → RAG

Hybrid 구조

Hybrid 구조

```
사용자 요청 → LLM: 전략 판단
    ↙                    ↘
RAG 검색               Tool 호출
(정적 문서)          (실시간 데이터)
    ↘                    ↙
         결과 통합
              ↓
         최종 답변
```

Hybrid 구조

```
사용자 요청 → LLM: 전략 판단
    ↙                    ↘
RAG 검색               Tool 호출
(정적 문서)          (실시간 데이터)
    ↘                    ↙
         결과 통합
              ↓
         최종 답변
```

> Hybrid는 복잡하고 비싸다.
> 먼저 "RAG만으로 안 되는가?", "Tool만으로 안 되는가?"를 확인하라.

구조 종합 비교

항목	MCP	RAG	Hybrid
실시간 데이터	지원	미지원	지원
외부 조작	가능	불가	가능
구현 복잡도	중간	중간	높음
운영 비용	API 비용	임베딩 비용	높음
지식 Q&A	제한적	강함	강함

구조 종합 비교

항목	MCP	RAG	Hybrid
실시간 데이터	지원	미지원	지원
외부 조작	가능	불가	가능
구현 복잡도	중간	중간	높음
운영 비용	API 비용	임베딩 비용	높음
지식 Q&A	제한적	강함	강함

선택 순서: RAG 검토 → MCP 검토 → Hybrid (최후 수단)

퀴즈: 구조 선택

고객 서비스 Agent:
제품 매뉴얼(PDF 500p)에서 답변 + 주문 현황 실시간 조회

A) MCP만 사용

B) RAG만 사용

C) Hybrid (RAG + MCP)

D) 단순 LLM 호출

퀴즈: 구조 선택

고객 서비스 Agent:
제품 매뉴얼(PDF 500p)에서 답변 + 주문 현황 실시간 조회

A) MCP만 사용

B) RAG만 사용

C) Hybrid (RAG + MCP)

D) 단순 LLM 호출

정답: **C** — 매뉴얼 검색(RAG) + 주문 조회(MCP Tool) 모두 필요

Session 4 실습

MCP vs RAG 구조 설계 비교

실습: 구조 설계 비교

실습: 구조 설계 비교

- **I DO (15분)**: 두 시나리오에 MCP/RAG/Hybrid 판단 과정 시연

실습: 구조 설계 비교

- **I DO (15분)**: 두 시나리오에 MCP/RAG/Hybrid 판단 과정 시연
- **WE DO (20분)**: Agent 후보에 구조 선택 + 체크리스트 적용

실습: 구조 설계 비교

- **I DO (15분)**: 두 시나리오에 MCP/RAG/Hybrid 판단 과정 시연
- **WE DO (20분)**: Agent 후보에 구조 선택 + 체크리스트 적용
- **YOU DO (25분)**: 최종 구조 결정 문서 작성

실습: 구조 설계 비교

- **I DO (15분)**: 두 시나리오에 MCP/RAG/Hybrid 판단 과정 시연
- **WE DO (20분)**: Agent 후보에 구조 선택 + 체크리스트 적용
- **YOU DO (25분)**: 최종 구조 결정 문서 작성

YOU DO 제출물 (`labs/agent-problem-definition/artifacts/`)

- 최종 구조 결정 (MCP/RAG/Hybrid)
- 선택 이유 3가지 이상
- 비선택 구조의 단점
- 예상 비용 구조

Day 1 정리

Day 1 정리

- **Session 1** : Pain → Task → Skill → Tool, Agent 패턴 3가지, RAG vs Agent

Day 1 정리

- **Session 1**: Pain → Task → Skill → Tool, Agent 패턴 3가지, RAG vs Agent
- **Session 2**: Token·Context·Hallucination, Zero/Few/CoT, Structured Output, 비용 최적화

Day 1 정리

- **Session 1**: Pain → Task → Skill → Tool, Agent 패턴 3가지, RAG vs Agent
- **Session 2**: Token·Context·Hallucination, Zero/Few/CoT, Structured Output, 비용 최적화
- **Session 3**: Sub-task 분해, Stateless/Stateful, IPO 명세, 예외 처리 전략

Day 1 정리

- **Session 1**: Pain → Task → Skill → Tool, Agent 패턴 3가지, RAG vs Agent
- **Session 2**: Token·Context·Hallucination, Zero/Few/CoT, Structured Output, 비용 최적화
- **Session 3**: Sub-task 분해, Stateless/Stateful, IPO 명세, 예외 처리 전략
- **Session 4**: MCP Tool 설계, RAG 구조, Hybrid 선택 기준

Day 1 정리

- **Session 1**: Pain → Task → Skill → Tool, Agent 패턴 3가지, RAG vs Agent
- **Session 2**: Token·Context·Hallucination, Zero/Few/CoT, Structured Output, 비용 최적화
- **Session 3**: Sub-task 분해, Stateless/Stateful, IPO 명세, 예외 처리 전략
- **Session 4**: MCP Tool 설계, RAG 구조, Hybrid 선택 기준

핵심 메시지

- Agent는 문제에서 시작한다. 기술에서 시작하지 않는다.
- 가장 단순한 구조를 먼저 시도한다.
- 실패 케이스를 설계 단계에서 정의한다.

Q&A

오늘 배운 내용 중 궁금한 점을 질문해주세요.

Day 2 예고: Agent 개발 기초 — 실제 코드로 Agent 구현